

Kernel-level DDoS Protection Using eBPF in IoT Networks

Kristina Živanović
The University of Belgrade
School of Electrical Engineering
Belgrade, Serbia
zk245024p@student.etf.bg.ac.rs

Pavle Vuletić
The University of Belgrade
School of Electrical Engineering
Belgrade, Serbia
pavle.vuletic@etf.bg.ac.rs

Abstract — This paper presents an in-kernel mechanism implemented using the Extended Berkeley Packet Filter for IoT system Denial of Service attack detection and prevention. The proposed mechanism detects attacks that exploit MQTT vulnerabilities and is executed in XDP (eXpress Data Path) of Linux-based kernel before the packet reaches the kernel socket buffers and before it enters the system network stack. Protection mechanisms are implemented on the message broker as the central point for all MQTT traffic, making it the most effective location to detect potential DoS attacks before they affect the connected IoT devices. Experimental results prove that the approach is lightweight and capable of detecting and preventing DoS attacks.

Keywords — MQTT, DoS, eBPF, XDP

I. INTRODUCTION

Internet of Things (IoT) systems present an increasing cybersecurity challenge due to their increased proliferation and the characteristic resource constraints their components have. [1]. One of the recent developments was the exploitation of new communication protocols for data transfer, like the Message Queuing Telemetry Transport (MQTT) protocol. Designed to be lightweight and efficient in environments where information needs to be exchanged in real time, MQTT is well-suited for IoT systems. Nevertheless, the protocol is exposed to various security risks, one of the most frequent being Denial of Service (DoS) attacks, which attracted a lot of recent attention from researchers. [2] At the moment of writing this paper, there are 170 recorded CVE (Common Vulnerabilities and Exposures) vulnerabilities related to MQTT and its implementations, with 56 and 76 recorded in 2023 and 2024, respectively. This indicates wider MQTT usage, an increased number of implementations, but also more vulnerability analyses. With the growing number of compromised IoT devices, DoS attacks have become increasingly devastating. In recent history, there have been well-known examples of DoS attacks using IoT devices. The most notable examples are the Mirai botnet attack in 2016, creating a botnets capable of generating traffic peaks of up to 1.2 Tbps and causing service disruption across many worldwide platforms such as Netflix, Twitter, and Reddit. [3] Also, the well-known Reaper attack, which appeared in 2017, was even more dangerous as it exploited security vulnerabilities in IoT device firmware, infecting devices such as CCTV cameras and routers. [4]

Common IoT protection measures include Intrusion Detection and Protection Systems (IDS/IPS), firewalls, and traffic filtering mechanisms, often combined with anomaly detection or machine learning-based approaches to identify and mitigate abnormal network behavior in real time [5]. While those defense strategies rely on resource-intensive methods, IoT systems demand more lightweight and efficient solutions due to strict latency requirements. To address these

challenges, this work explores the use of eBPF for implementing lightweight in-kernel packet filtering and enabling real-time responses with minimal overhead.

The paper is structured as follows. In Section II, the MQTT protocol and its vulnerabilities are presented, then eBPF is presented in Section III, where it is described in accordance with the implementation use-case within this work. Section IV describes logic for in-kernel DDoS protection, then Section V presents results. Lastly, Section VI provides the paper's conclusion and future research.

II. MQTT PROTOCOL

MQTT protocol is a lightweight protocol for real-time data exchange within Machine-to-Machine or IoT contexts, standardized by OASIS and ISO. This protocol is widely used in IoT networks and is designed to be optimal when it comes to resource utilization in resource-limited sensor devices. [1] [5] MQTT is an application-level protocol based on the publish/subscribe model, where messages are exchanged between nodes in the network via a central message broker. The broker is responsible for keeping records of clients, monitoring which topics they are subscribed to, and forwarding messages accordingly [6]. MQTT is primarily used for exchanging sensor data, control instructions, and event notifications, where devices publish sensor readings, status updates, or control commands that are delivered in real time to subscribed clients, which then display this information, notify other parts of the system, or trigger appropriate actions. Message brokers enable decoupling of devices and services, allowing heterogeneous devices to communicate efficiently without direct connections. While brokers may be deployed on local networks or in the cloud, they are typically not directly exposed to the public Internet for security reasons. [7] Common examples of message brokers include RabbitMQ and Mosquitto, which are open-source, lightweight, and widely used in smart home systems, industrial monitoring, and sensor networks.

MQTT is designed with an emphasis on the speed of communication between nodes in the network, optimal consumption of processor resources, memory, and energy, rather than protection against possible attacks [8]. It does not include encryption, but it is possible to use it encapsulated within SSL/TLS protocols. However, encapsulating MQTT into TLS/SSL brings problems with resource load when TLS is used on end devices and is sometimes omitted. Another method is encryption of the MQTT payload on the application level by implementing proprietary ways of key exchange and algorithm implementation. [9]

In the past few years, researchers have found several attack scenarios using the MQTT protocol: CONNECT flooding, Delay CONNECT flooding, Invalid Subscription flooding, CONNECT flooding with WILL payload, and TCP SYN flooding [8]. Authors in [10] [11] presented detection methods

based on Machine Learning. In this paper, the CONNECT flooding attack will be discussed.

Figure 1 shows the connection establishment method between a client and a message broker through a simple exchange of CONNECT and CONNACK messages. [12] The session control packet consists of a fixed and variable header and the payload (Figure 2). MQTT connect packet has a control header equal to 0x10; the variable header consists of protocol name ('MQTT'), protocol level byte (version of MQTT protocol used), connect flag byte, and two bytes of keep alive value. Payload contains clientID, username, and password. All user-configurable strings are prefixed with two bytes indicating their length. MQTT CONNECT attacks occur during the connection phase to the MQTT broker and after the TCP session has been established. [13] If the client sends a large number of CONNECT packets in succession without closing the previously established session with the broker, the broker's resources will be overloaded by storing data on half-open connections, which may lead to a decrease in the performance of the broker itself or to disabling the service, which is essentially a DoS attack. [1]

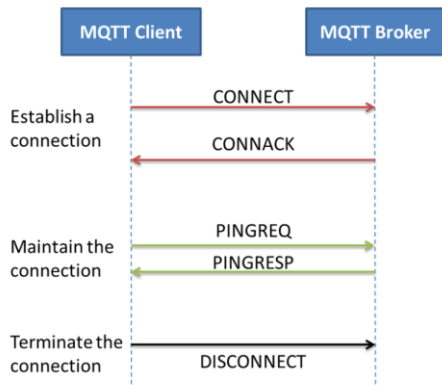


Figure 1 Session state in the MQTT protocol [12]

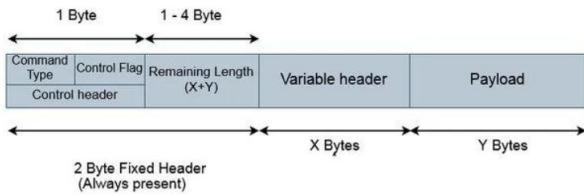


Figure 2 MQTT packet format [14]

III. EBPF

Extended Berkeley Packet Filter (eBPF) is an instruction set and runtime environment integrated into the Linux kernel, providing a way to extend or modify kernel functionalities without rewriting actual Linux kernel code. It allows dynamic interaction with kernel functions throughout programs written in restricted C, compiled into eBPF bytecode, and then executed in the kernel space. This is crucial because avoiding transfers to the user space significantly reduces processing latency. Nowadays, applications of this mechanism comprise a wide range of use-cases such as network monitoring, network traffic handling, load balancing, operating system insight, etc [15].

Since version 3.18 of the Linux kernel, eBPF has been introduced as a built-in functionality. It includes an in-kernel execution environment with 64-bit registers and the use of a JIT (just-in-time) compiler. In addition, it supports calling eBPF programs from different network layers, so-called sockets, which allows customized packet filtering in different layers of the network stack. [16]

A significant feature of eBPF is information storage structures represented as maps. Depending on the usage and implementation, eBPF programs can be attached to parts of the kernel where they will be executed every time a certain event happens. These programs do not maintain state information between multiple executions, which is overcome by maps that allow state storage and transfer. They are generic structures and can also be used to transfer information between the kernel and the user space.

Furthermore, an important process is the verification of programs that are executed in the kernel. Executing an irregular program in the kernel, even if it is within a kernel virtual machine, can lead to the incapacitation of the operating system kernel, which is why the BPF verifier is built into the kernel itself. This verifier is a program that attempts to detect irregularities in code execution by analyzing the user program, preventing cycles, checking that folders are not deleted before the program that uses them ends, preventing undefined program flows, etc. [17]. In this paper, the eBPF program is implemented to run in Xpress Data Path (XDP), which means it will be executed every time a packet arrives on a network interface. XDP is a part of the network driver that allows access to packets by reading them from the network interface, allowing actions to be taken before any memory is allocated for a particular packet in the kernel socket buffers. When used for DoS/DDoS attack prevention purposes, packets can be dropped before loading the kernel's TCP stack or a user application. [17]

IV. DDoS DETECTION AT KERNEL LEVEL

This paper discusses a method for MQTT CONNECT DoS/DDoS attack detection implemented within the eBPF packet filter at the kernel level. The eBPF packet filter is run on the message broker. The program is integrated within the simplified IoT system, which consists of a message publisher/sender, a message receiver, and a message broker (Figure 3). The IoT sender publishes data on a certain topic to the message broker every second. The IoT receiver is subscribed to the same topic on which the sender is publishing in order to receive the sender's data. A DoS attacker is a PC that sends a large number of MQTT CONNECT packages in order to produce a Denial of Service on the broker.

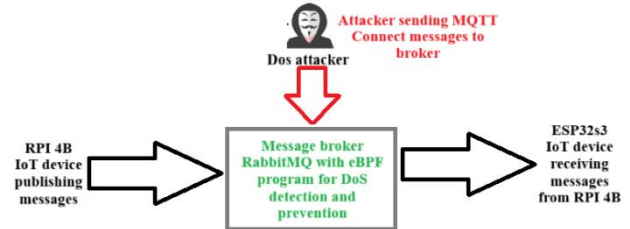


Figure 3 Scheme of the IoT system used

The first experiment was creating a DoS attack on the broker without ebpf-based protection in order to baseline the load under which a regular end device (IoT sender) is unable to publish data to the broker, and consequently, the subscriber (IoT receiver) is unable to receive data. The second experiment introduced the eBPF kernel-based protection in order to prove that this kind of protection reduces or eliminates the possibility of Denial of Service in this specific testing environment.

The eBPF program developed for the DoS/DDoS prevention purposes in this paper is running within the XDP, intercepting every packet that arrives on the incoming interface. The filtering algorithm is given in Figure 4. The

basic structure for storing information across different executions of an eBPF program is a map of type `BPF_MAP_TYPE_HASH`, which holds up to 1024 source IP addresses as keys, and a separate structure with count and time of last packet received from these IP addresses. This map is a shared resource among every execution of this code. Also, there is another map structure of the same type that holds black listed IP addresses, or IPs from which defined limits for denial of service detection were exceeded. This structure holds the IP address as key and the time when a certain IP was black listed as value. After a packet from this source IP arrives, and more than 30s have passed from the recorded time in this map, the record is removed from the map and the packet is further processed by this script, otherwise, the packet is discarded with `XDP_DROP` and is not subject to any further processing.

The implemented filter has two tunable parameters: connection threshold and time window, where the threshold presents the maximum number of `CONNECT` attempts from a single IP within the configured time window that the broker host could tolerate without significant degradation in CPU or memory performance under our test conditions. In the presented experiments, we used 1000 connections per 0.005 seconds as a rule of thumb, but this is tunable to balance detection sensitivity with system performance. These limits were derived from stress tests that measured CPU, memory, and packet-loss behavior at varying flood intensities on the available IoT equipment.

```
on_packet_receive(packet):
    if packet.protocol != TCP:
        return XDP_PASS

    if packet.tcp.dest_port != MQTT_PORT:
        return XDP_PASS
    if packet.tcp.payload_first_byte != MQTT_CONNECT:
        return XDP_PASS

    src_ip = packet.ip.src

    if src_ip in BLACKLIST:
        if current_time - BLACKLIST[src_ip].timestamp < 30s:
            return XDP_DROP
        else:
            remove BLACKLIST[src_ip]

    if src_ip not in CONNECTION_MAP:
        CONNECTION_MAP[src_ip] = { count: 1, last_time: current_time }
        return XDP_PASS

    entry = CONNECTION_MAP[src_ip]
    delta_time = current_time - entry.last_time

    if entry.count > CONNECT_THRESHOLD and delta_time < TIME_WINDOW:
        BLACKLIST[src_ip] = { timestamp: current_time }
        return XDP_DROP

    if delta_time < TIME_WINDOW:
        entry.count += 1
    else:
        entry.count = 1
        entry.last_time = current_time
        CONNECTION_MAP[src_ip] = entry

    return XDP_PASS
```

Figure 4 Pseudo-code of the implemented filtering algorithm

Briefly, the filtering algorithm functions as follows. Upon packet arrival, the filter inspects the protocol and port to select MQTT traffic. Then it looks for MQTT `CONNECT` packets, while others are passed to the network stack (`XDP_PASS`). Then it checks the extracted IP address, whether the address is in the black list. If that is the case, the time when that IP was black listed is compared with the current time, and if 30s have not passed, the packet is dropped with the `XDP_DROP` command, otherwise, the record from the „black list“ is deleted, and further processing of this packet in the ebpf filter continues. `CONNECTION_MAP` is searched to find an

existing record with this IP as a key. If the address is not in the map, create a new element with a connection count of 1, insert it into the map, and pass the packet. If there were already connections from that IP address, compare the number of connections in the record of the map with the `CONNECT_THRESHOLD`, and the time the last detected packet arrived with the time the packet currently being processed arrived. If the number of connections from that IP address is greater than the threshold defined as a constant, and the time difference between the last and currently processed packet is less than the defined `TIME_WINDOW`, drop the packet and put the extracted IP address in the black list, where it stays for at least 30 seconds. If these conditions are not met, it is necessary to update the map with an increased number of connections or 1 if more than `TIME_WINDOW` time has passed.

V. RESULTS

Tests were conducted using variable attack throughput intensities, measured as the number of established MQTT connections in two minutes. Longer attack durations were also performed, but it was confirmed that attack success rate metrics did not significantly change beyond the 2-minute mark. The computer that was running the RabbitMQ broker has two CPU cores, Intel i5 9th Gen, and 8GB of RAM. IoT sender logic was implemented on a Raspberry Pi 4B, and IoT receiver logic was implemented on an ESP32s3 device.

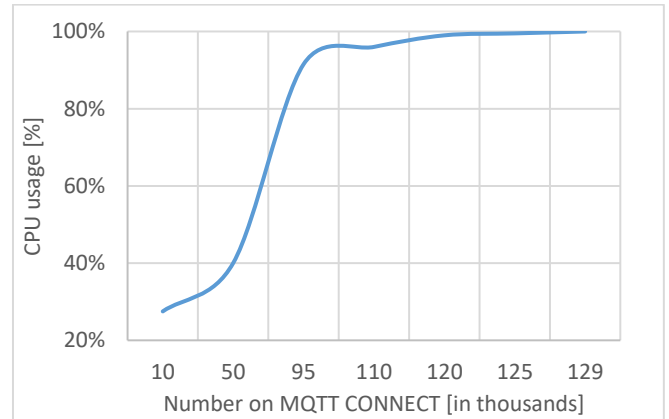


Figure 5 Relation between CPU usage of broker and the Number of MQTT Connect packages without ebpf-based protection

The eBPF filter was implemented in the C programming language. The number of MQTT `CONNECT` packets that were sent from the attacker was: 10.000, 50.000, 95.000, 110.000, 120.000, 125.000, and 129.000. At these levels, measurable changes in the broker host's susceptibility to DoS were observed, manifested as increased CPU utilization, packet loss rates, and reduced success of MQTT publish operations. In Figure 5, the correspondence between the number of connection requests and the maximum CPU usage, within 2 minutes, of the host where the RabbitMQ broker was executing, is shown. This was measured without ebpf-based DoS protection. Figure 5. shows that with around 100.000 connection requests, the broker's processor becomes fully saturated, and this is the case when the DoS attack fully occupies the processor.

This is further analysed in Figure 6, which shows the relation between the number of connection requests sent by the attacker and the percentage of successfully published messages by regular end users of the IoT system discussed in

this paper. Above 50.000 requests, the percentage of published messages starts to fall.

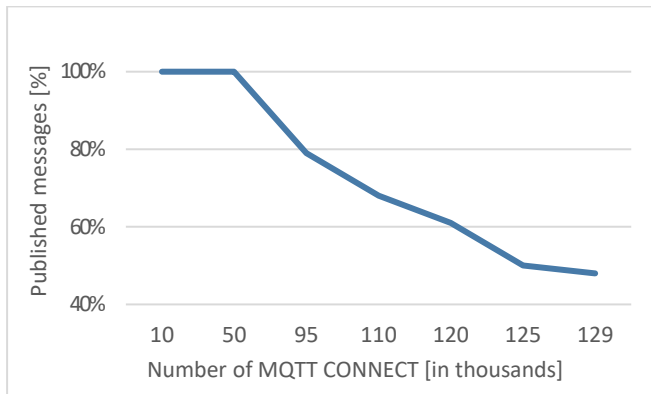


Figure 6 Relation between the percentage of published messages and the number of MQTT CONNECT requests without ebpf-based protection

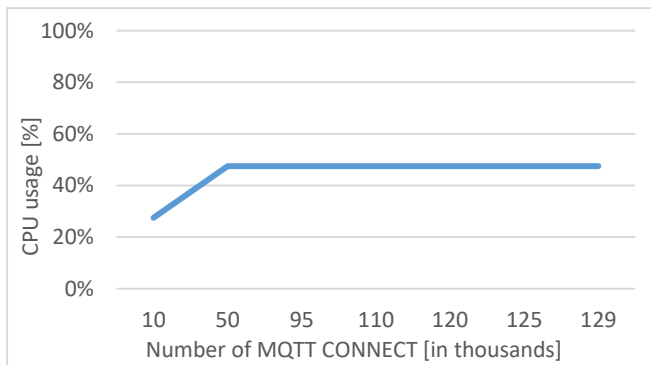


Figure 7 The Relation between CPU usage of the broker and the number of MQTT Connect packages with ebpf-based protection on the broker

Finally, after including the ebpf filter into the kernel and attaching it to the corresponding interface, significant improvements were observed (Figure 7). During this experiment, 100% of messages published from the RPi end device were successfully received on the ESP32s3 end device regardless of the attack intensity, while the maximum CPU usage did not exceed 95%, even with a larger number of DoS requests (above 100.000). This proves that the system managed to efficiently block the attack and keep some spare processor time for other activities.

VI. CONCLUSION

The implemented eBPF-based packet filtering algorithm has met expectations regarding DoS detection and prevention within the specified IoT environment, demonstrating that in-kernel filtering is an efficient approach that provides performance improvements compared to user-space solutions. By processing packets directly in the kernel space, the latency is reduced and CPU load is minimized, proving the benefits of the presented approach for resource-constrained systems. Our future research will include exploring variations to the algorithm to include various patterns of DoS attacks, including Distributed DoS (DDoS), and to analyze the impact of using variable map sizes on system performance and efficiency.

VII. REFERENCES

- [1] L. V. Y. N.Arton, "Detection of TCP and MQTT-Based DoS/DDoS Attacks on MUD IoT Networks," *Electronics*, vol. 1653, p. 14, 2025.
- [2] A. Alatrani, L.Sikos, M.Johnstone, P.Szewczyk and J.Kang, "DoS/DDoS-MQTT-IoT: A dataset for evaluating intrusions in IoT networks using the MQTT protocol," *Computer Networks*, vol. 231, 2023.
- [3] P.Kumari and A. Jain, "A comprehensive study of DDoS attacks over IoT network and their countermeasures," *Computers & Security*, vol. 127, 2023.
- [4] S. R. M.Salim and J.H.Park, "Distributed denial of service attacks and its defenses in IoT:A survey," *The Journal of Supercomputing*, 2019.
- [5] S.Kaganutmath and N.Cholli, "Enabling Robust Security in MQTT-Based IoT Networks with Dynamic Resource-Aware Key Sharing," in *4th International Conference on Evolutionary Computing and Mobile Sustainable Networks*, 2025.
- [6] F. A. a. T. Alhazmi, "Secure Data Distribution Architecture in IoT Using MQTT," *MDPI*, pp. 1-13, 2023.
- [7] E.Kirdan, P.Horvath and M.Pahl, "Work-in-Progress: Slow Denial of Service Attack on MQTT-Based IoT," in *IEEE International Black Sea Conference on Communications and Networking*, Istanbul, Turkey, 2023.
- [8] N. T. K. S. M. Swain, "Identifying communication sequence anomalies to detect DoS attacks against MQTT," *Computers & Security*, vol. 157, 2025.
- [9] S.Alhabi, D.Bell and W.Awad, "Application Layer Security: MQTT perspective with TLS Implementation and Analysis," in *International Conference on IT Innovation and Knowledge Discovery*, 2024.
- [10] H.Zeghida, M.Boulaiche and R.Chikh, "Detection of DoS Attacks in MQTT Environment," *Communications in Computer and Information Science*, pp. 129-140, 2023.
- [11] N. F. Syed, "IoT-MQTT based denial of service attack modelling and detection," Edith Cowan University, 2020.
- [12] S. S. K.Ravi, "MQTT based home automation system using ESP8266," in *2016 IEEE Region 10 Humanitarian Technology Conference*, Agra, India, 2016.
- [13] Oasis, "MQTT Version 5," 07 03 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>.
- [14] "MQTT Packet Format," OpenLabPro, [Online]. Available: <https://openlabpro.com/mqtt-packet-format/>. [Accessed 12 09 2025].
- [15] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacifico and Santos, "Fast Packet Processing with eBPF and XDP: Concepts, Code,Challenges, and Applications," *ACM Digital Library*, vol. 53, no. 1, pp. 1-36, 2020.
- [16] Y. C. M. L. S. Baidya, "eBPF-based Content and Computation-aware Communication for Real-time Edge Computing," *arXiv:1805.02797v1*, vol. 1, 2018.
- [17] D. Chodhury, "XDP-Programmable Data Path in the Linux Kernel," *Programming*, vol. 43, no. 1, 2018.